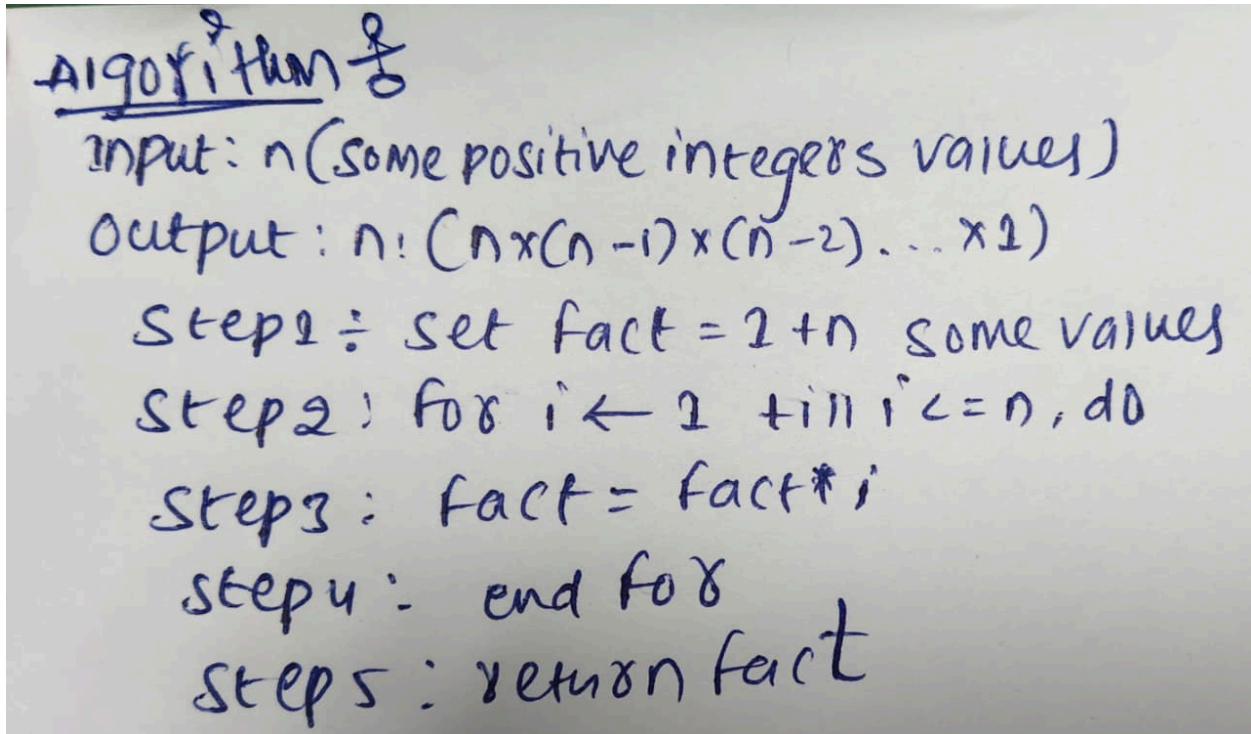

PRACTICAL-4

AIM: Implementation and Time analysis of factorial program using iterative and recursive method.

Factorial Using Iterative Method

ALGORITHM:



Algorithm

Input: n (some positive integers values)

Output: $n! (n \times (n-1) \times (n-2) \dots \times 1)$

Step 1: Set $fact = 1 + n$ some values

Step 2: for $i \leftarrow 1$ till $i \leq n$, do

Step 3: $fact = fact * i$

Step 4: end for

Step 5: return $fact$

CODE:

```
#include <iostream>
using namespace std;

class Factorial
{
public:
    int fact(int n)
    {
        int result = 1;

        for (int i = 1; i <= n; i++)
        {
```

```
        result = result * i;
    }

    return result;
}

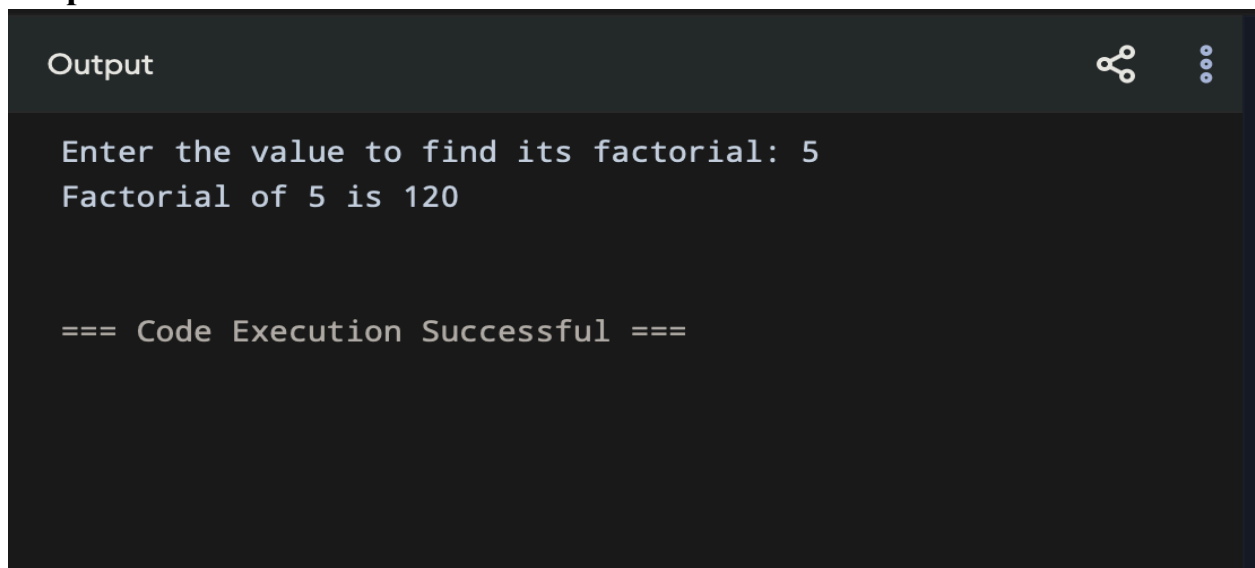
int main()
{
    int n,result;
    Factorial obj;

    cout << "Enter the value to find its factorial: ";
    cin >> n;

    result = obj.fact(n);

    cout << "Factorial of " << n << " is " << result << endl;

    return 0;
}
```

Output:

```
Output

Enter the value to find its factorial: 5
Factorial of 5 is 120

=== Code Execution Successful ===
```



Time Complexity:

Time complexity :-

for (int i=1; i<=n; i++) $\rightarrow n$

{ result = result * i;

}

$T(n) = n \times 1 \Rightarrow T(n) = O(n)$

Best case :-

for (i=1; i<=n; i++) $\rightarrow n$

{ result = result * i; $\rightarrow 1$

}

$T(n) = n \times 1$

$T(n) = O(n)$

Average case :-

for (i=1; i<=n; i++) { $\rightarrow n$

result = result * i; $\rightarrow 1$

}

$T(n) = n \times 1$

$T(n) = O(n)$

worst case :-

for (int i=0; i<=n; i++) { $\rightarrow n$

result = result * i;

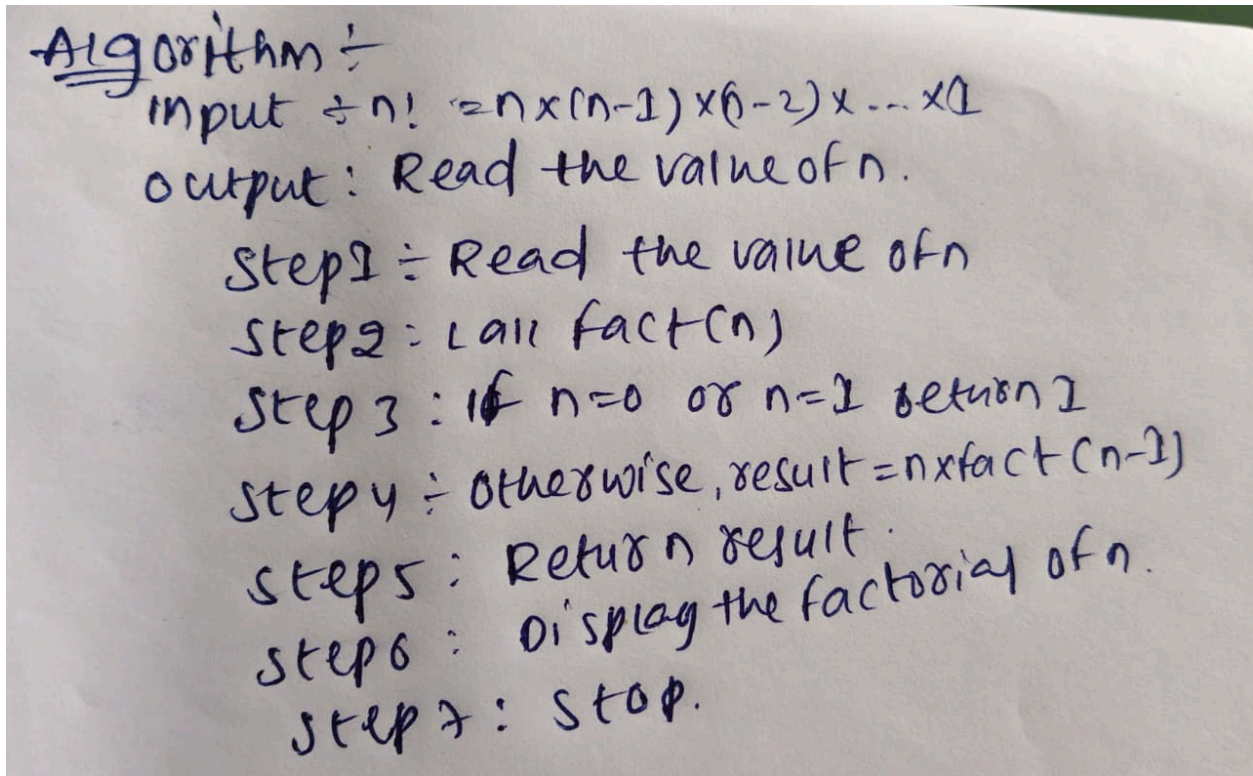
}

$T(n) = n \times 1$

$T(n) = O(n)$

Factorial Using Recursive Method:

Algorithm



Algorithm :-
input :- $n! = n \times (n-1) \times (n-2) \times \dots \times 1$
output: Read the value of n .
Step 1 :- Read the value of n
Step 2 :- Call fact(n)
Step 3 :- If $n=0$ or $n=1$ return 1
Step 4 :- Otherwise, result = $n \times \text{fact}(n-1)$
Step 5 :- Return result.
Step 6 :- Display the factorial of n .
Step 7 :- Stop.

Code:

```
#include <iostream>
```

```
using namespace std;
```

```
class Factorial
```

```
{
```

```
public:
```

```
    int factr(int n)
```

```
    {
```

```
        if (n == 0 || n == 1) {  
            return 1;  
        }  
        else  
        {  
            int result = n * factr(n -  
1);  
            return result;  
        }  
    }  
};  
  
int main()  
{  
    int n, result;  
  
    // Create an object of the class  
    Factorial obj;  
  
    cout << "Enter the value to  
find its factorial: ";  
  
    cin >> n;
```



```
result = obj.factr(n);  
  
cout << "Factorial of " << n  
  
<< " is " << result << endl;  
  
return 0;  
  
}
```

Output:

Output

```
Enter the value to find its factorial: 6  
Factorial of 6 is 720
```

```
=== Code Execution Successful ===
```

TIME COMPLEXITY:



Time complexity :

```
int fact(int n) {
    if (n == 0 || n == 1) → 1
    return 1; → 1
    else
        result = n * fact(n-1) n-1
    return result;
}
```

$T(n) = \max(n, n-1)$
 $= n$
 $T(n) = O(n)$

Best case : fact(int n) {
 if (n == 0 || n == 1) → 1
 { return 1;
 }
 }
 $T(n) = 1 \times 1$
 $T(n) = O(1)$

worst case :

```
= fact(int n) {
    if (n == 0 || n == 1) → n
    { return 1; → 1
    } else {
        result = n * fact(n-1) → n-1
        return result; → 1
    }
}
```

$T(n) = n \times 1$
 $T(n) = O(n)$

Average case :

```
fact(int n) {
    if (n == 0 || n == 1) → n
    { return 1;
    } else {
        result = n * fact(n-1) → n-1
        return result; → 1
    }
}
```

$T(n) = n \times 1$
 $T(n) = O(n)$

Conclusion:

The factorial program was successfully implemented using both iterative and recursive methods. Both methods produce the same result, but the recursive method uses function calls and stack space. The time complexity of both methods is $O(n)$, making them efficient for calculating factorials.